AgriGuard: Model Development Journey

Technical Report on Phases 1-5: Environment, Preprocessing, Architecture, Debugging & Validation

Project Development Presentation Deck
Prepared By: Navin badode and AI Assistant
Focus: Local CNN Pipeline, Double-Preprocessing Bug Diagnostics, SQLite Log & Dynamic Advisory

Development Phases Overview

The development lifecycle of the complete AgriGuard platform spans 10 distinct phases:

Phase	Phase Name	Key Milestones & Achievements
Phase 1	Environment Setup	Resolved TensorFlow, NumPy, and h5py library compatibility issues in a clean environment.
Phase 2	Dataset Preparation	Selected 15 PlantVillage categories (Tomato, Potato, Pepper classes).
Phase 3	Dataset Verification	Resolved nested folders, generated statistics (20,638 images), and implemented TF dataloaders.
Phase 4	Model Training	Built a MobileNetV2 transfer learning model with data augmentation and a Dropout(0.3) head.
Phase 5	Prediction System	Resolved a double preprocessing inference bug that caused inaccurate blight classifications.
Phase 6	Web Routing & Control	Developed Flask web endpoints for user logins, image uploads, history scans, and bot dialogs.
Phase 7	Database Persistence	Structured SQLite schemas for user credentials management and detailed diagnostic log histories.
Phase 8	Multilingual Localization	Implemented static and dynamic translation dictionaries for Marathi, Tamil, Telugu, Hindi, and English.
Phase 9	Smart Advisories	Integrated weather-smart irrigation metrics, soil nutrient planning algorithms, and context-aware chat.
Phase 10	QA & Offline Scripts	Created diagnostics tools (verify_model.py, predict.py) to validate weights compilation and local inferences.

Phase 1 & 2: Environment Setup & Dataset Selection

Phase 1: Environment Setup

- **Libraries:** Installed TensorFlow and basic scientific computing packages.
- **Compatibility Diagnostics:** Resolved version mismatch issues between TensorFlow API boundaries, NumPy array shapes, and h5py model file serialization.
- **Outcome:** Established a clean, reproducible Python environment ensuring successful execution of import statements.

Phase 2: Dataset Selection (PlantVillage Subset)

- **Crop Scope:** Focused on Pepper, Potato, and Tomato crops.
- **Categories Included (15 Total):**

- *Pepper:* Bacterial spot, Healthy

- *Potato:* Early blight, Late blight, Healthy

- *Tomato:* Bacterial spot, Early blight, Late blight, Leaf mold, Septoria spot, Spider mites, Target spot, Yellow leaf curl, Mosaic virus, Healthy

Phase 3: Dataset Verification & Statistics

Directory Structure Discovered & Fixed:

- **Nested Folder Issue:** The root dataset directory path was verified and corrected to bypass a nested `PlantVillage/PlantVillage` directory, establishing the inner folder as the primary `DATASET_PATH`.
- **Loading Mechanism:** Configured TensorFlow's standard `image_dataset_from_directory` utility to handle label parsing.
- **Label Alignment:** Verified alphabetical file name loading order to prevent category index mismatches.

Final Dataset Split Statistics:

- **Total Loaded Images:** 20,638 images
- **Total Category Classes:** 15 classes
- **Training Subset (80% Split):** 16,511 images
- **Validation Subset (20% Split):** 4,127 images
- **Verification:** Verified loading and class mappings in memory to ensure stable training iterations.

Phase 4: Model Architecture & Preprocessing

Architecture Specifications:

- **Base Feature Extractor:** Transfer learning with a pre-trained **MobileNetV2** base (weights frozen on ImageNet).
- **Regularization:** Configured a **Dropout(0.3)** layer before the output density head to prevent over-fitting.
- **Output Head:** A Dense layer with Softmax activation containing 15 classification units.
- **Data Augmentation Layers:** Sequentially applies Random Flip (horizontal), Random Rotation (10%), and Random Zoom (10%).

Model Layer Flow Diagram:

```
Input Image (RGB, 224 x 224)
↓
Data Augmentation (Flip, Rotate, Zoom)
↓
MobileNetV2 Preprocessing (Rescaling from [0,255] to [-1,1])
↓
MobileNetV2 Base Backbone (Frozen weights)
↓
GlobalAveragePooling2D
↓
Dropout(0.3)
↓
Dense Softmax Output (15 Classes Category probabilities)
```

Phase 5: Diagnostics & Troubleshooting Inference Bug

Bug Discovery & Anomaly:

- **The Symptom:** During initial pipeline testing, the model predicted `Tomato_Late_blight` with ~99% confidence for nearly every uploaded image, regardless of actual crop species or leaf health.
- **Root Cause Analysis:** Discovered a double-preprocessing bug: `preprocess_input()` was executed twice during inference.

1. Once inside the built-in preprocessing layers of the trained Keras model.
2. A second time in the external input pipeline of the predictor script (`predict.py`).

The Correction & Outcome:

- **Resolution:** Removed the external `preprocess_input()` normalization step from the prediction file (`predict.py` / `app.py`).
- **Strategy:** Kept the input scaling sequence (`[0, 255]` to `[-1, 1]`) encapsulated strictly inside the model architecture itself.
- **Final Inference Workflow:**

```
Load Image → Resize (224x224) → Convert to Array → model.predict()
```

- **Outcome:** Model predictions aligned correctly with actual classes, restoring true validation accuracy.

Model Evaluation & Validation Results

Key Validation Performance Metrics:

- **Validation Accuracy: 89.6%**
- **Validation Loss: 0.3048**
- **Inference Stability:** Verified by compiling outputs across 4,127 evaluation images.
- **Significance:** Confirmed that the transfer learning layers successfully generalized feature extraction (edges, color gradients, necrosis patterns) without overfitting.

Training Context & Performance Strategy:

- **Optimizer:** Adam optimizer with Sparse Categorical Crossentropy loss.
- **Callback Operations:** EarlyStopping monitored validation loss to halt training, while ModelCheckpoint saved only the best-performing iteration to file.
- **Class Names File:** Exported as `class_names.txt` to guarantee translation indexes map correctly to Flask application views.

Core Platform Features: AI Diagnostics & Support

1. Dual-Engine Diagnostics Backend

- **Local Offline CNN:** Fast inference running on the local server for common crops (Tomato, Potato, Pepper), minimizing data costs.
- **Cloud AI Vision Fallback:** Uses Llama 4 Vision to identify complex pests and other crops (Apple, Wheat, Cotton, Mango, Grape, Citrus, etc.).
- **Advisory Engine:** Generates customized 21-day timeline guides detailing chemical/organic treatments and spray dosages.

2. Multilingual AgriBot Chatbot

- **Context Inheritance:** The chat model automatically inherits the scan context (detected disease, cause, symptoms, organic/chemical treatment) to provide customized follow-up advice.
- **Multilingual Dialog:** Auto-detects input and replies in the farmer's preferred language (Marathi, Tamil, Telugu, Hindi, English).
- **Accessibility:** Encouraging, clear, and farmer-friendly advice without overly technical jargon.

Core Platform Features: Weather Advisory & Soil Planner

3. Weather-Smart Intelligence

- **Local Climate Parsing:** Integrates local temp, humidity, and rain metrics to evaluate active fungal blight or pest risks.
- **Advisory Actions:** Prompts actions like irrigation adjustment (delaying due to rain) and foliar spray wash-away alerts.

4. Growth-Stage Fertilizer Calculator

- **Targeted Scheduling:** Custom fertilizer scheduler based on crop type, growth phase (vegetative, flowering, fruiting), and soil structure.
- **Deficiency Solutions:** Addresses observed leaf anomalies with organic and mineral NPK ratios.

5. User Accounts & Scan Logs

- **SQLite Log Base:** Secure registration and salted/hashed passwords (`agri_guard.db`) mapping to unique profiles.
- **History Tracking:** Persists details (symptoms, cause, chemical/organic treatment, photos) for farmers to review past records.

6. Farmer-Friendly Localization

- **Multilingual UI:** Static strings and dynamic recommendations map instantly to **Marathi, Tamil, Telugu, Hindi, and English**.

Operational Pipeline & Production Deployment

The verified model is integrated into a Flask web application, featuring command line maintenance scripts:

1. Start the simple training pipeline (from scratch):

```
python train.py
```

2. Run the automated model check tool:

```
python verify_model.py
```

3. Execute image diagnosis predictions directly:

```
python predict.py <path_to_image>
```

4. Run the Flask Web Application locally (with SQLite logs & localized UI):

```
python app.py
```